ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТ 1
ГАРЫН АВЛАГА

Үйлдлийн систем /F.CSM302/

Сэдэв: ”Designing a Thread Pool. Programming Project 1 (7-р бүлэг), Worker activity”

Хуваарилалтын код: IV-2-9

Хичээл заасан багш: Г.Ганбат /F.CS10/ .....

Гүйцэтгэсэн баг: Ц.Номинжин /B241940137/ .....
Е.Айсауле /B221940407/ .....
Б.Балжинням /B231940033/ .....
М.Бат-Орших /B200930061/ .....
Г.Хулан /B241940027/ .....
Э.Энхсайхан /B242270097/ .....

Улаанбаатар хот

2026 он

ГАРЧИГ

НЭГ. *Хэрэглэгчийн заавар* 2

1.1 Программын зорилго 2

1.2 Дэмжих орчин 2

1.3 Windows: WSL2 2

1.4 Windows: virtual machine 2

1.5 Суулгах, нээх. 3

1.6 Дэлгэцийн бүтэц. 3

1.7 Оролт ба тохиргоо 4

1.8 Ажиллуулах, зогсоох, reset 5

1.9 Дүрслэлийн тэмдэг, өнгө, төлөв 6

1.10 Жишээ. 6

1.11 Сервергүй горим 9

1.12 Түгээмэл алдаа ба шийдэл 9

ХОЁР. *Operational заавар* 11

2.1 Repository clone 11

2.2 OS, compiler/runtime, version 11

2.3 Dependency суулгах 11

2.4 Build ба Run 11

2.5 Үндсэн тестийн өгөгдөл 12

2.6 Нэмэлт boundary ба error тест 12

2.7 Хүлээгдсэн үр дүн 13

2.8 Known issues 13

НЭГ. ХЭРЭГЛЭГЧИЙН ЗААВАР

1.1 Программын зорилго

Сурах бичгийн 7-р бүлгийн thread pool-ийг C хэл дээр pthreads, mutex, POSIX semaphore-оор хэрэгжүүлсэн. Client thread task-уудыг queue-д хийж, worker thread-үүд нь queue-ээс авч гүйцэтгэнэ. Browser дээрх дэлгэц нь **бодит C программын** үйлдлийг (submit, татгалзал, эхлэх, дуусах, shutdown) worker бүрийн төлөв, queue болон timeline хэлбэрээр харуулна. Дэлгэц зөвхөн харуулдаг; бүх синхрончлол C core дотор явагдана.

1.2 Дэмжих орчин

- Linux (Ubuntu 22.04+, Debian, Fedora), macOS (Apple Silicon / Intel), Windows дээр **WSL2 Ubuntu** дотор.
- Browser: Chrome, Edge, Firefox, Safari-ийн сүүлийн хувилбар.
- Windows дээр шууд ажиллахгүй (pthreads, POSIX semaphore ашигладаг): **WSL2** эсвэл **virtual machine** доторх Ubuntu ашиглана.

1.3 Windows: WSL2

1. PowerShell-ийг администратороор нээж `wsl --install -d Ubuntu`, компьютераа дахин асаана.
2. Ubuntu терминалд compiler, Python суулгана:

```
sudo apt update && sudo apt install -y build-essential git python3
```

1. ”Суулгах, нээх” хэсгийн командуудыг Ubuntu дотор ажиллуулна. Windows-ийн browser дээр http://localhost:8000 шууд нээгдэнэ.

1.4 Windows: virtual machine

VirtualBox, VMware эсвэл UTM дээр Ubuntu 22.04+ суулгаж, WSL2-ийн 2-р алхмын адил `build-essential`, `python3` суулгана.

- VM доторх browser-оос: `make ui`, http://localhost:8000.
- Windows (host) browser-оос: сервер анхдагчаар зөвхөн VM дотроос хүсэлт хүлээн авдаг тул `python3 tools/server.py --host 0.0.0.0` гэж ажиллуулаад `http://<VM-ийн IP>:8000` нээнэ (IP-г `hostname -I`-ээр харна).
- VM-ийн сүлжээ Bridged эсвэл Host-only байна; NAT бол 8000 портыг forward хийнэ.

1.5 Суулгах, нээх

```
git clone https://github.com/NominjinNomio/F.CSM302-BD_IV-2-9-threadpool.git
cd F.CSM302-BD_IV-2-9-threadpool
make          # C программыг build хийнэ
make ui       # http://localhost:8000
```

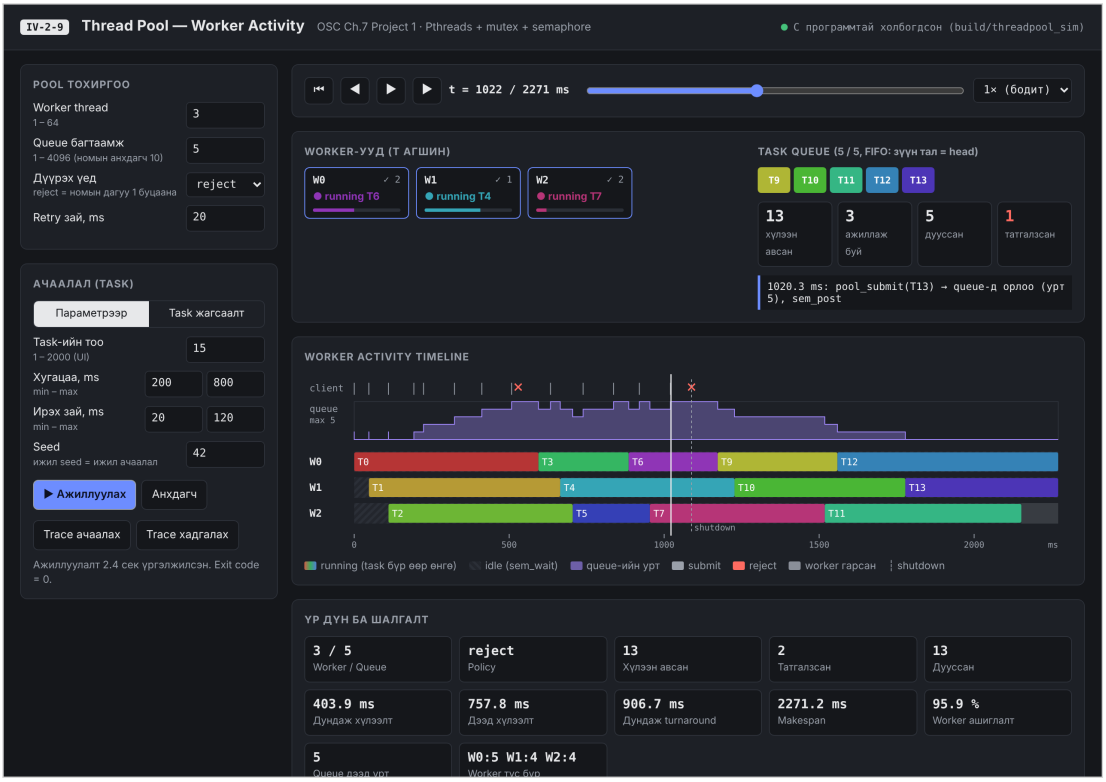
Дараа нь browser-т **http://localhost:8000**-ийг нээнэ. Баруун дээд буланд ногоон цэг ба ”C программтай холбогдсон” гэж гарвал бэлэн. Хуудас нээгдэхэд анхдагч тохиргоогоор нэг удаа автоматаар ажиллана.

1.6 Дэлгэцийн бүтэц

Дэлгэц зүүн талын тохиргоо, баруун талын дүрслэлээс бүрдэнэ (Хүснэгт 1, Зураг 1).

Хүснэгт 1. Дэлгэцийн хэсгүүд

Хэсэг	Байрлал	Юу хийх вэ
Pool тохиргоо	зүүн дээд	worker, queue, дүүрэх үеийн бодлого, retry зай
Ачаалал (task)	зүүн доод	Параметрээр эсвэл Task жагсаалтаар task үүсгэх; Ажиллуулах, Анхдагч, Trace ачаалах/хадгалах
Playback мөр	баруун дээд	Reset, өмнөх/дараагийн үйлдэл, тоглуулах, хугацааны гулсуур, хурд
Worker-ууд, Task queue	дунд	t агшин дахь worker бүрийн төлөв, queue-ийн нүд, тоолуур, үйлдлийн тайлбар
Worker activity timeline	дунд	worker бүрийн хугацааны мөр, client мөр, queue-ийн урт
Үр дүн ба шалгалт	доор	хэмжүүрүүд, 8 шалгалт, C программын команд ба гаралт
Үйлдлийн бүртгэл	хамгийн доор	үйлдэл бүрийн хүснэгт; мөр дээр дарж тэр агшин руу очно



Зураг 1. Дэлгэцийн ерөнхий харагдац

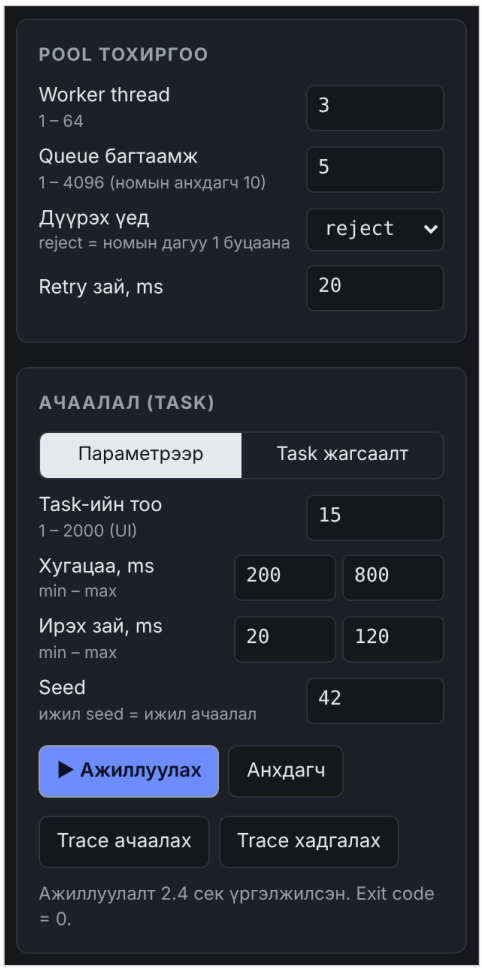
1.7 Оролт ба тохиргоо

Тоо бүрийг шууд бичиж эсвэл сумаар өөрчилнө. Хүрээнээс гадуур утга оруулбал Ажиллуулах дарахад улаан талбарт алдааны тайлбар гарна (Зураг 10). **Параметрээр** горимд task-уудыг seed-ээс санамсаргүй үүсгэнэ; **Task жагсаалт** горимд мөр бүрт `arrival_ms duration_ms` бичнэ (#-ээс хойш тайлбар, arrival өсөх дарааллаар). **Бэлэн тест**-ээс `basic.txt`, `burst.txt`, `fifo.txt`, `mixed_long_short.txt` сонгож болно.

Хүснэгт 2. Оролт ба тохиргоо

Талбар	Утга	Хүрээ	Анхдагч
Worker thread	pool дахь worker thread-ийн тоо	1-64	3
Queue багтаамж	тойрог массив queue-ийн хэмжээ	1-4096	5
Дүүрэх үед	reject: `pool_submit` 1 буцааж task хаягдана; retry: client хүлээгээд дахин оролдоно	reject / retry	reject
Retry зай (мс)	retry горимд дахин оролдох зай	1-10000	20
Task-ийн тоо	үүсгэх task-ийн тоо	1-2000	15
Хугацаа (мс)	task бүрийн хугацаа энэ мужаас санамсаргүй	1-60000	200-800

Талбар	Утга	Хүрээ	Анхдагч
Ирэх зай (мс)	дараалсан task-уудын зай; 0 = бүгд зэрэг	0-60000	20-120
Seed	ижил seed = ижил ачаалал	≥ 0	42



Зураг 2. Тохиргоо

1.8 Ажиллуулах, зогсоох, reset

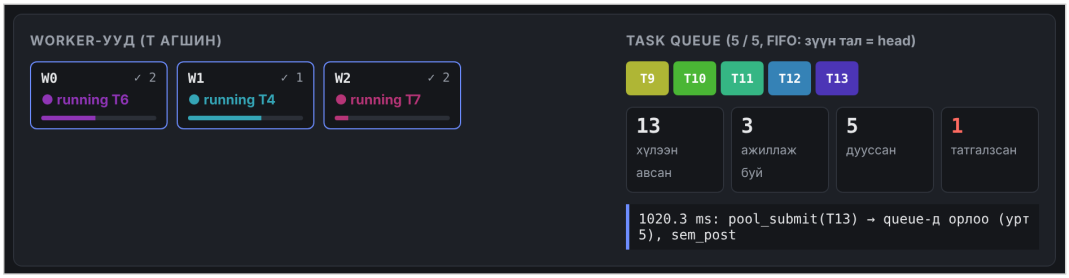
- **Ажиллуулах**: тохиргоогоор C программыг бодитоор ажиллуулна (task-уудын нийт хугацаа шиг үргэлжилнэ), дараа нь автоматаар тоглуулна.
- **▶ / ‖** (Space): тоглуулах, түр зогсоох. **◀ / ▶** (← →): өмнөх, дараагийн үйлдэл рүү алхах.
- **Гулсуур**, timeline дээр **дарах эсвэл чирэх**, бүртгэлийн мөр дээр дарах: тухайн агшин руу очно.
- **Хурд**: 0.25×, 0.5×, 1× (бодит хугацаа), 2×, 4×, 10×.
- **|◀ Reset**: эхний агшин руу буцна. **Анхдагч**: бүх тохиргоог анхны утганд оруулна.
- **Trace хадгалах**: тухайн ажиллалтыг JSON файлаар татна. **Trace ачаалах**: хадгалсан trace-ийг дахин нээнэ.
- **Серверийг зогсоох**: терминалд Ctrl+C.

1.9 Дүрслэлийн тэмдэг, өнгө, төлөв

Worker карт. Worker бүр нэг карт: нэр (W0, W1…), одоогийн төлөв, явцын шугам, баруун дээд буланд дуусгасан task-ийн тоо (✓ n).

- ● **running Tk** (өнгөт, хүрээ цэнхэр): worker Tk task-ийг гүйцэтгэж байна (*running*).
- ○ **idle — sem_wait** (саарал): worker semaphore дээр ажил хүлээж байна (*idle*).
- ■ **exited** (бүдэг): shutdown-ы дараа worker гарсан. ✓ n нь тухайн worker-ийн *completed* task-ийн тоо.

Task queue. Хүлээж буй task-ууд өөрийн өнгөөр, зүүн талд нь хамгийн түрүүнд авагдах (head). Тасархай хүрээтэй нүд = сул зай. Доор нь хүлээн авсан, ажиллаж буй, дууссан, татгалзсан тоо, түүний доор тухайн агшны үйлдлийн тайлбар (жишээ нь ”W1 sem_wait-аас сэрж T10-г dequeue хийв”).



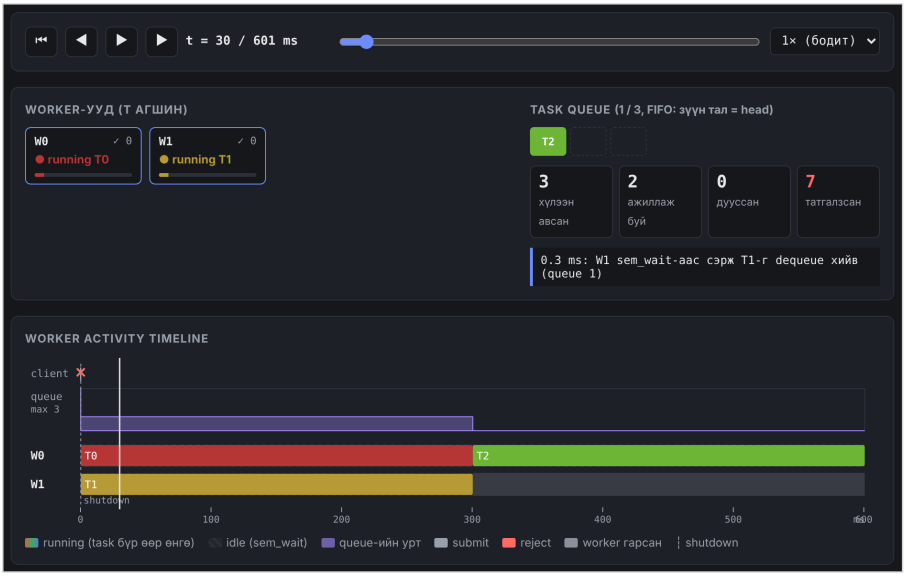
Зураг 3. Worker карт ба task queue

Хүснэгт 3. Worker activity timeline-ийн тэмдэг

Тэмдэг	Утга
өнгөт зурвас Tk	worker Tk task-ийг гүйцэтгэсэн хугацаа; task бүр өөр өнгөтэй, queue-д ч мөн тэр өнгөөр харагдана
судалтай саарал	idle: worker `sem_wait` дээр хүлээж байна
бараан бүс	worker гарсан (shutdown-ы дараа)
нил ягаан шатлалт график	queue-ийн урт (0 … багтаамж)
client мөр: саарал |	`pool_submit` амжилттай (0)
client мөр: улаан ✕	queue дүүрсэн, `pool_submit` 1 буцаасан
тасархай босоо зураас	`pool_shutdown()` дуудагдсан агшин
хар босоо шугам	одоогийн агшин (playhead)

1.10 Жишээ

Жишээ 1. Queue дүүрэх (Task жагсаалт → burst.txt, W=2, Q=3, reject). 10 task нэгэн зэрэг ирэх тул queue хурдан дүүрнэ: 3-5 task хүлээн авагдаж, үлдсэн нь client мөрөнд улаан ✕ болно. Төгсгөлд `VERIFY OK`.



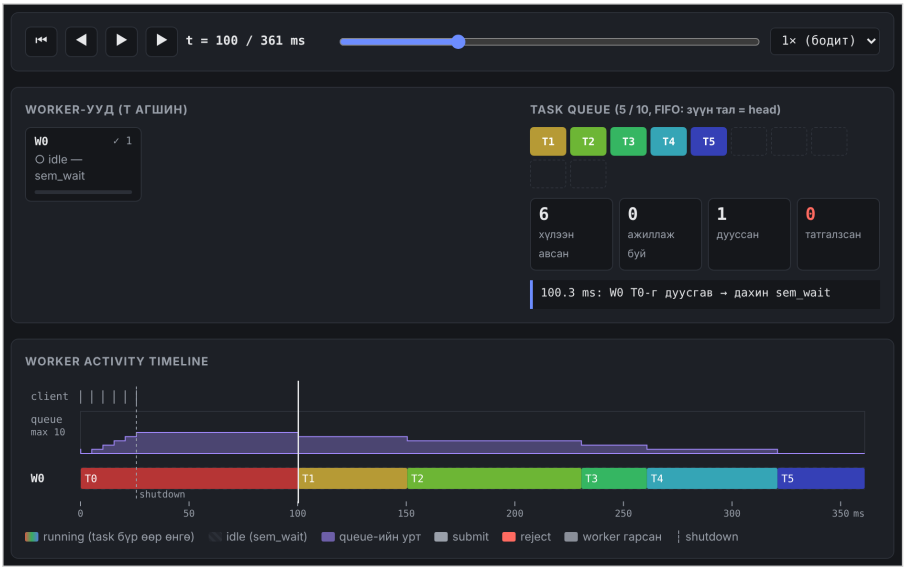
Зураг 4. Жишээ 1. Queue дүүрэх

Жишээ 2. Retry горим (Жишээ 1-ийн тохиргоо, Дүүрэх үед: retry). Client дүүрсэн queue-д 20 мс тутам дахин оролдох тул ✕ олон боловч 10 task бүгд гүйцэтгэгдэнэ, нийт хугацаа ≈ 1500 мс.



Зураг 5. Жишээ 2. Retry горим

Жишээ 3. FIFO-г алхамчлах (fifo.txt, W=1, Q=10). Ажиллуулаад |◀ дарж, ▶ (⇥) товчоор нэг нэгээр алхана. Task-ууд T0, T1, T2… дарааллаар эхэлж, queue-ийн зүүн талаас авагдана.



Зураг 6. Жишээ 3. FIFO, алхамчилсан

Жишээ 4. Worker-ийн тооны нөлөө (Параметрээр: 12 task, хугацаа 100-300, ирэх зай 0-0, queue 64, seed 1). Worker-ийг 1, дараа нь 4 болгож ажиллуулаад timeline болон ”Makespan”-ийг харьцуулна.



Зураг 7. Жишээ 4. Дээр 1 worker, доор 4 worker

Жишээ 5. Үйлдлийн бүртгэл. Бүртгэлийн мөр бүр нэг үйлдэл; одоогийн мөр цэнхэрээр, ирээдүйн мөрүүд бүдгээр харагдана. Мөр дээр дарахад тэр агшин руу очно.



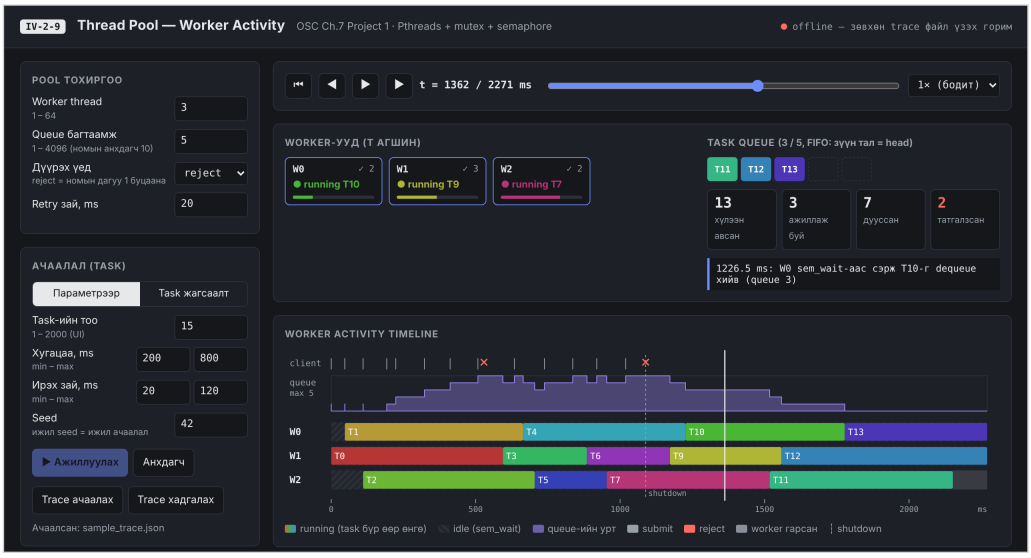
ҮЙЛ ЯВДЛЫН БҮРТГЭЛ (мөр дээр дарж тухайн агшинд очно)

#	t, ms	төрөл	worker	task	queue
0	0.00	POOL_INIT			0
1	0.16	SUBMIT		T0	1
2	0.17	START	W0	T0	0
3	47.43	SUBMIT		T1	1
4	47.55	START	W1	T1	0
5	110.60	SUBMIT		T2	1
6	110.75	START	W2	T2	0
7	192.75	SUBMIT		T3	1
8	223.87	SUBMIT		T4	2
9	323.66	SUBMIT		T5	3
10	411.83	SUBMIT		T6	4
11	507.99	SUBMIT		T7	5
12	529.20	REJECT		T8	5

Зураг 8. Жишээ 5. Үйлдлийн бүртгэл

1.11 Сервергүй горим

`ui/index.html`-ийг файлаар шууд нээвэл баруун дээд буланд ”offline — зөвхөн trace файл үзэх горим” гарч, Ажиллуулах идэвхгүй болно. **Trace ачаалах**-аар `examples/sample_trace.json` эсвэл өөрийн хадгалсан trace-ийг нээж бүх дүрслэлийг ашиглаж болно.



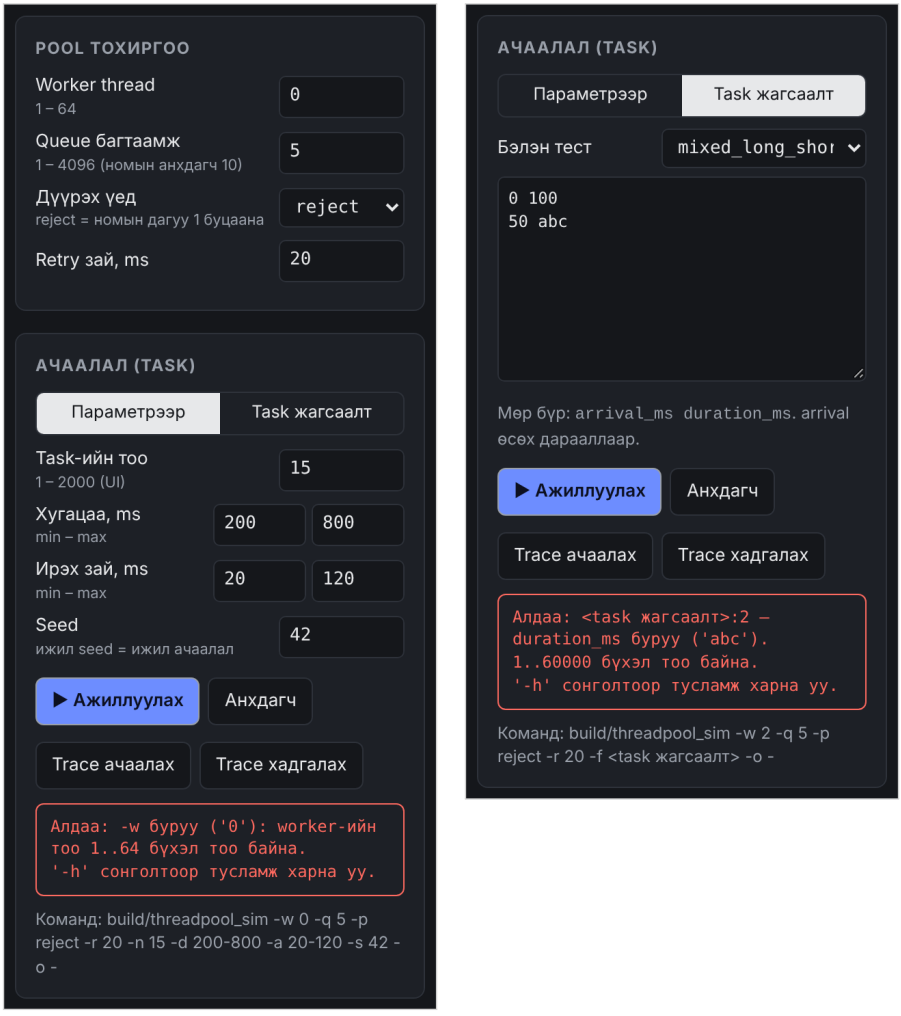
Зураг 9. Сервергүй горимд хадгалсан trace үзэж буй нь

1.12 Түгээмэл алдаа ба шийдэл

Хүснэгт 4. Түгээмэл алдаа

Алдаа	Шалтгаан	Шийдэл
`-w буруу ('0'): worker-ийн тоо 1..64`	хүрээнээс гадуур утга	тайлбарт заасан мужид оруулна (Зураг 10)
`<task жагсаалт>:2 — duration_ms буруу`	жагсаалтын 2-р мөрөнд тоо биш утга	мөр бүр яг 2 бүхэл тоо
`arrival_ms өсөхгүй дараалаар`	ирэх хугацаа эрэмбэгүй	arrival-аар эрэмбэлнэ

Алдаа	Шалтгаан	Шийдэл
`Тооцоолсон хугацаа ~N сек`	интерфейсийн 60 с-ийн хязгаар	task/хугацааг багасгах, worker нэмэх эсвэл CLI ашиглах
`offline — зөвхөн trace`	сервер ажиллаагүй / файлаар нээсэн	`make ui`, дараа нь http://localhost:8000
`build/threadpool_sim алга`	build хийгээгүй	`make`
`make: cc: No such file`	C compiler байхгүй	Ubuntu/WSL: `sudo apt install build-essential`; macOS: `xcode-select --install`
`Address already in use`	8000 порт завгүй	`python3 tools/server.py --port 8080`
Ижил тохиргоо өөр timeline	OS scheduler-ийн nondeterminism	алдаа биш; Trace хадгалахаар тогтооно



Зураг 10. Буруу оролт: worker = 0 (зүүн), task жагсаалтад тоо биш утга (баруун)

ХОЁР. OPERATIONAL ЗААВАР

2.1 Repository clone

```
git clone https://github.com/NominjinNomio/F.CSM302-BD_IV-2-9-threadpool.git
cd F.CSM302-BD_IV-2-9-threadpool
git checkout v1.0      # 9-р долоо хоногт нийтэлсэн хувилбар
```

2.2 OS, compiler/runtime, version

Хүснэгт 5. Туршсан орчин

Орчин	OS	Compiler	Python
Хөгжүүлэлт, тест	Ubuntu 24.04.5 LTS, x86_64	gcc 13.3.0	3.13.18
GitHub Actions	ubuntu-latest	gcc	3
GitHub Actions	macos-latest	Apple clang	3

Шаардлага: C11 compiler (gcc ≥ 9 эсвэл clang ≥ 10) + `make`, Python 3.8 эсвэл түүнээс дээш, орчин үеийн browser. Python-д pip сан, интерфейст интернет (CDN) шаардлагагүй.

2.3 Dependency суулгах

```
# Ubuntu / Debian / WSL2
sudo apt update && sudo apt install -y build-essential python3
# Fedora
sudo dnf install -y gcc make python3
# macOS
xcode-select --install
```

2.4 Build ба Run

```
make                        # build/threadpool_sim, build/client
make ui                     # UI сервер, http://localhost:8000
make client                 # номын client.c жишээ
make run                    # жишээ run -> build/trace.json
# core-ийг шууд ажиллуулах:
./build/threadpool_sim -w 3 -q 5 -n 15 -d 200-800 -a 20-120 -o trace.json
./build/threadpool_sim -w 2 -q 3 -f tests/burst.txt -p retry
./build/threadpool_sim -h    # бүх сонголт
```

Exit код: **0** амжилттай, **2** буруу оролт, **3** ажиллах үеийн алдаа (санах ой, thread үүсгэх), **4** дотоод шалгалт унасан.

2.5 Үндсэн тестийн өгөгдөл

```
make test
```

- `./build/client`: номын жишээ, 5 хос тооны нийлбэр (15, 3, 300, 0, 42).
- `tests/basic.txt`: 10 task, W=3, Q=10 → хүлээн авсан 10, татгалзсан 0, зэрэг ≤ 3.
- `tests/burst.txt`: 10 × 300 мс, t=0; W=2, Q=3 → reject үед 3-5 хүлээн авна, retry үед 10.
- `tests/fifo.txt`: 6 task, W=1 → START дараалал T0…T5.
- `tests/mixed_long_short.txt`: 1 × 1500 мс + 7 × 100 мс, W=3.

Trace-ийг бие даан шалгах: `python3 tests/verify_trace.py trace.json --expect accepted=15`.

2.6 Нэмэлт boundary ба error тест

```
# queue 1: татгалзал гарна
./build/threadpool_sim -w 1 -q 1 -n 6 -d 100 -a 0
# дээд 64 worker
./build/threadpool_sim -w 64 -q 64 -n 64 -d 50 -a 0
# 500 task stress
./build/threadpool_sim -w 8 -q 16 -n 500 -d 1-3 -a 0 -p retry -r 1
make tsan     # ThreadSanitizer
```

Хүснэгт 6. Буруу оролтын тестийн бодит гаралт (exit 2)

Тест	Программын мэдээлэл
`err_workers_0`	Алдаа: -w буруу (’0’): worker-ийн тоо 1..64 бүхэл тоо байна.
`err_workers_65`	Алдаа: -w буруу (’65’): worker-ийн тоо 1..64 бүхэл тоо байна.
`err_queue_0`	Алдаа: -q буруу (’0’): queue-ийн багтаамж 1..4096 байна.
`err_queue_text`	Алдаа: -q буруу (’abc’): queue-ийн багтаамж 1..4096 байна.
`err_tasks_0`	Алдаа: -n буруу (’0’): task-ийн тоо 1..10000 байна.
`err_dur_0`	Алдаа: -d буруу (’0’): ’MIN’ эсвэл ’MIN-MAX’, 1..60000 ms.
`err_dur_order`	Алдаа: -d буруу (’500-100’): MIN нь MAX-аас их байж болохгүй.
`err_gap_neg`	Алдаа: -a буруу (’-5’): ’MIN’ эсвэл ’MIN-MAX’, 0..60000 ms.
`err_policy`	Алдаа: -p буруу (’fifo’): ’reject’ эсвэл ’retry’ байна.
`err_missing_val`	Алдаа: ’-w’ сонголтод утга өгөөгүй байна.
`err_unknown_opt`	Алдаа: Танигдаагүй сонголт ’-x’.
`err_file_missing`	Алдаа: ’tests/no_such_file.txt’ файлыг нээж чадсангүй (No such file or directory).

Тест	Программын мэдээлэл
`err_file_number`	Алдаа: tests/bad_number.txt:2 — arrival_ms буруу (’abc’). 0..3600000 бүхэл тоо байна.
`err_file_sorted`	Алдаа: tests/bad_unsorted.txt:2 — arrival_ms өсөхгүй дарааллаар байна (’50’). Эрэмбэлэгдсэн байх ёстой.
`err_file_columns`	Алдаа: tests/bad_columns.txt:1 — мөр бүр ’arrival_ms duration_ms’ гэсэн 2 тоотой байх ёстой.
`err_file_dur0`	Алдаа: tests/bad_duration.txt:1 — duration_ms буруу (’0’). 1..60000 бүхэл тоо байна.
`err_file_empty`	Алдаа: ’tests/bad_empty.txt’ файлд нэг ч task алга.

2.7 Хүлээгдсэн үр дүн

- `make test`: төгсгөлд `Нийт: 29 PASS, 0 FAIL`, exit 0.
- Ажиллуулалт бүрийн stderr-ийн төгсгөлд 8 мөр `[ OK ]` ба `VERIFY: OK`.
- `make tsan`: ThreadSanitizer-ийн `WARNING` гарахгүй, `VERIFY: OK`.
- Хугацааны утгууд (хүлээлт, makespan) машин бүрт хэдэн мс-ээр зөрнө; хүлээн авсан/татгалзсан тоо, FIFO, зэрэгцээний дээд хэмжээ өөрчлөгдөхгүй (burst-ийн 3-5 мужаас бусад).
- GitHub Actions: build-and-test (linux, macos) ногоон.

2.8 Known issues

- macOS нэргүй POSIX semaphore (`sem_init`) дэмждэггүй тул `dispatch_semaphore` ашиглана; macOS-ийг зөвхөн CI-д шалгасан.
- Windows дээр шууд ажиллахгүй; WSL2 эсвэл VM ашиглана.
- Worker-ийн хуваарилалт ба burst үеийн хүлээн авах тоо ажиллуулалт бүрт бага зэрэг өөр (OS scheduler).
- Интерфейсээр нэг ажиллуулалт ≤ 2000 task, ≈ 60 с. Илүүг CLI-ээр ажиллуулж, trace-ийг ачаалж үзнэ (их task-тай trace зурахад browser удааширч болно).
- Retry горимд нэг client thread хүлээдэг тул дараагийн task-уудын ирэлт хойшилно.
- Маш богино (< 1 мс) task-д trace-ийн mutex хэмжилтэд нөлөөлнө.